C++

Java

Python

JavaScript

```
class Object
    inheritance
polymorphism
  encapsulation
```

Object-Oriented Programming

The Complete Guide to OOP Concepts

C++ | Java | Python | JavaScript

Mohammad Faiz

Year 2026 - Multi-Language Edition

```
+-----------------+
| Class |
+-----------------+
| - attributes |
| + methods() |
+-----------------+
```

Copyright Notice

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use: No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content for any purpose (commercial or non-commercial).

Personal Use Only: Only the original author Mohammad Faiz has the authority to use this content in any project or application.

Attribution: If you discuss or reference the concepts, you must provide full credit to the original author Mohammad Faiz.

Multi-Language Edition

This book includes code examples in:

- C++
- Java
- Python
- JavaScript

Table of Contents

Chapter 1: Introduction to OOP 5

1.1 What is OOP? .....

1.2 OOP vs Procedural Programming .....

Chapter 2: Classes and Objects 7

2.1 Defining a Class .....

2.2 Creating Objects .....

Chapter 3: Encapsulation 10

3.1 Access Modifiers .....

3.2 Getters and Setters .....

Chapter 4: Inheritance 14

4.1 Types of Inheritance .....

4.2 Method Overriding .....

Chapter 5: Polymorphism 19

5.1 Compile-time Polymorphism .....

5.2 Runtime Polymorphism .....

Chapter 6: Abstraction 22

6.1 Abstract Classes .....

6.2 Interfaces .....

Chapter 7: Constructors and Destructors 26

7.1 Types of Constructors .....

Chapter 8: Static and Final 30

Chapter 9: Exception Handling 34

Chapter 10: Design Patterns 37

Chapter 11: SOLID Principles 43

Chapter 12: Interview Preparation.....48

Appendix A: Language Syntax Comparison.....52

Appendix B: OOP Cheat Sheet.....55

Chapter 1: Introduction to Object-Oriented Programming

Welcome to the world of Object-Oriented Programming (OOP)! This chapter will introduce you to the fundamental concepts that revolutionized software development. OOP helps us organize code in a way that mirrors real-world objects and their interactions.

1.1 What is OOP?

Object-Oriented Programming is a programming paradigm that organizes software design around data (objects) rather than functions and logic. An object is a self-contained unit that contains both data (attributes) and procedures (methods) to manipulate that data.

The Four Pillars of OOP

- Encapsulation: Bundling data and methods that work on that data within one unit
- Inheritance: Creating new classes from existing ones
- Polymorphism: Ability to take multiple forms
- Abstraction: Hiding complex implementation details

Real-World Analogy

Think of a car. A car has properties (color, brand, speed) and behaviors (start, stop, accelerate). In OOP, the car is an object with attributes (data) and methods (functions).

1.2 OOP vs Procedural Programming

Aspect	Procedural	OOP
Organization	Functions and procedures	Objects and classes
Data	Separate from functions	Bundled with methods
Reusability	Limited	High (inheritance)
Security	Low	High (encapsulation)
Maintenance	Difficult	Easier

1.3 Benefits of OOP

- Modularity: Code is organized into self-contained objects
- Reusability: Inheritance allows code reuse
- Maintainability: Easier to update and debug
- Scalability: Easier to build large applications
- Security: Encapsulation protects data

1.4 Practice Questions

Question 1: What are the four pillars of OOP?

Answer: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Question 2: Why is OOP better than procedural programming?

Answer: OOP provides better code organization, reusability through inheritance, data protection through encapsulation, and easier maintenance for large projects.

Chapter 2: Classes and Objects

A Class is a blueprint or template for creating objects. An Object is an instance of a class. Think of a class as a cookie cutter and objects as the cookies made from it.

2.1 Defining a Class - Multi-Language

C++

```
class Car {
private:
    string brand;
    string model;
    int year;

public:
    // Constructor
    Car(string b, string m, int y)
        { brand = b;
        model = m;
        year = y;
    }

    void displayInfo() {
        cout << brand << " " << model << " " << year;
    }
};
```

Java

```
public class Car
    { private String
    brand; private String
    model; private int
    year;

    // Constructor
    public Car(String b, String m, int y)
        { brand = b;
        model = m;
        year = y;
    }

    public void displayInfo() {
        System.out.println(brand + " " + model + " " + year);
    }
```

Python

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year

    def display_info(self):
        print(f"{self.brand} {self.model} {self.year}")
```

2.2 Creating Objects

C++

```
// Creating objects
Car car1("Toyota", "Camry", 2023);
Car car2("Honda", "Civic", 2022);

// Using object methods
car1.displayInfo();
car2.displayInfo();
```

Java

```
// Creating objects
Car car1 = new Car("Toyota", "Camry", 2023);
Car car2 = new Car("Honda", "Civic", 2022);

// Using object methods
car1.displayInfo();
car2.displayInfo();
```

Python

```
# Creating objects
car1 = Car("Toyota", "Camry", 2023)
car2 = Car("Honda", "Civic", 2022)

# Using object methods
car1.display_info()
car2.display_info()
```

2.3 The 'this' Keyword / 'self'

The 'this' keyword (C++, Java) or 'self' (Python) refers to the current object instance.

Java

```
public class Person {
```

```
    private String name;

    public Person(String name) {
        this.name = name;  // this refers to the instance variable
    }
}
```

Python

```
class Person:
    def __init__(self, name):
        self.name = name  # self refers to the instance
```

2.4 Practice Questions

Question 1: Create a Student class with name, age, and grade attributes.

Python

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"{self.name}, Age: {self.age}, Grade: {self.grade}")

# Create student object
student = Student("Alice", 20, "A")
student.display()
```

Question 2: What is the difference between a class and an object?

Answer: A class is a blueprint/template that defines properties and behaviors. An object is an instance of a class created from that blueprint.

Chapter 3: Encapsulation

Encapsulation is the bundling of data and methods that operate on that data within a single unit (class), and restricting access to some of the object's components.

3.1 Access Modifiers

Access modifiers control the visibility of class members:

Modifier	Class	Package	Subclass
private	Yes	No	No
protected	Yes	Yes	Yes
public	Yes	Yes	Yes

C++

```
class BankAccount {
private:      // Only accessible within class
    double balance;

protected:    // Accessible in class and derived classes
    string accountType;

public:       // Accessible from anywhere
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    double getBalance()
        { return balance;
    }
};
```

Java

```
public class BankAccount {
    private double balance;        // Only within class
    protected String accountType;  // Class + subclasses
    public String ownerName;       // Anywhere

    public void deposit(double amount) {
```

```
        if (amount > 0)
            { balance +=
            amount;
        }
    }

    public double getBalance()
        { return balance;
    }
```

Python

```
class BankAccount:
    def __init__(self):
        self.__balance = 0  # Private (name mangling)
        self._account_type = "" # Protected (convention)
        self.owner_name = ""   # Public

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self._balance
```

3.2 Getters and Setters

Getters and setters provide controlled access to private data.

Java

```
public class Person
    { private String
    name; private int
    age;

    // Getter
    public String getName()
        { return name;
    }

    // Setter
    public void setName(String name)
        { this.name = name;
    }

    // Getter with validation
    public int getAge() {
```

```
    }

    public void setAge(int age) {
        if (age >= 0 && age <= 150)
            { this.age = age;
        }
    }
}
```

Python

```
class Person:
    def  __init__(self):
        self._name = ""
        self._age = 0

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        self._name = value

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if 0 <= value <= 150:
            self._age = value
```

3.3 Practice Questions

Question 1: Why is encapsulation important?

Answer: Encapsulation provides data hiding, protects internal state, allows validation in setters, and makes code more maintainable by controlling how data is accessed and modified.

Question 2: Create a Rectangle class with private width and height, and public getter/setter methods.

Python

```
class Rectangle:
    def __init__(self, width=0, height=0):
```

```
        self.__width = width
        self.__height = height

    @property
    def width(self):
        return self.__width

    @width.setter
    def width(self, value):
        if value > 0:
            self.__width = value

    @property
    def height(self):
        return self.__height

    @height.setter
    def height(self, value):
        if value > 0:
            self.__height = value

    def area(self):
        return self.__width * self.__height
```

Chapter 4: Inheritance

Inheritance is a mechanism where a new class (child/subclass) derives properties and behaviors from an existing class (parent/superclass). It promotes code reusability.

4.1 Basic Inheritance - Multi-Language

C++

```
// Base class
class Animal {
protected:
    string name;

public:
    void eat() {
        cout << name << " is eating." << endl;
    }

    void sleep() {
        cout << name << " is sleeping." << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << name << " says: Woof!" << endl;
    }
};
```

Java

```
// Base class
class Animal {
    protected String name;

    public void eat() {
        System.out.println(name + " is eating.");
    }

    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}
```

```
// Derived class
class Dog extends Animal
    { public void bark() {
        System.out.println(name + " says: Woof!");
    }
}
```

Python

```
# Base class
class Animal:
    def __init__(self):
        self.name = ""

    def eat(self):
        print(f"{self.name} is eating.")

    def sleep(self):
        print(f"{self.name} is sleeping.")

# Derived class
class Dog(Animal):
    def bark(self):
        print(f"{self.name} says: Woof!")
```

4.2 Types of Inheritance

1. Single Inheritance

One child class inherits from one parent class.

2. Multiple Inheritance

```
// C++ supports multiple inheritance
class FlyingCar : public Car, public Airplane {
    // Inherits from both Car and Airplane
};
```

Python

```
# Python supports multiple inheritance
class FlyingCar(Car, Airplane):
    def __init__(self):
        Car.__init__(self)
        Airplane.__init__(self)
```

3. Multilevel Inheritance

```
class GrandParent { };
class Parent : public GrandParent { };
class Child : public Parent { };
// Child inherits from Parent, which inherits from GrandParent
```

4. Hierarchical Inheritance

Multiple classes inherit from a single parent class.

4.3 Method Overriding

Method overriding allows a child class to provide a specific implementation of a method that is already defined in its parent class.

Java

```
class Animal {
    public void makeSound()
        { System.out.println("Animal makes a
        sound");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound()
        { System.out.println("Dog barks: Woof!");
    }
}

class Cat extends Animal {
    @Override
    public void makeSound()
        { System.out.println("Cat meows: Meow!");
    }
```

Python

```
class Animal:
    def make_sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def make_sound(self):
        print("Dog barks: Woof!")
```

```
class Cat(Animal):
    def make_sound(self):
        print("Cat meows: Meow!")
```

4.4 The 'super' Keyword

The 'super' keyword is used to call parent class methods or constructors.

Java

```
class Animal {
    protected String name;

    public Animal(String name)
        { this.name = name;
    }
}

class Dog extends Animal
    { private String breed;

    public Dog(String name, String breed)
        { super(name);  // Call parent
        constructor this.breed = breed;
    }
}
```

Python

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed
```

4.5 Practice Questions

Question 1: Create a Vehicle base class and Car/Motorcycle derived classes.

Python

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
```

```
    def start(self):
        print(f"{self.brand} {self.model} is starting.")

class Car(Vehicle):
    def __init__(self, brand, model, num_doors):
        super().__init__(brand, model)
        self.num_doors = num_doors

    def honk(self):
        print("Beep beep!")

class Motorcycle(Vehicle):
    def __init__(self, brand, model, has_sidecar):
        super().__init__(brand, model)
        self.has_sidecar = has_sidecar

    def wheelie(self):
        print("Doing a wheelie!")
```

Question 2: What is the difference between method overloading and method overriding?

Answer: Method overloading is having multiple methods with the same name but different parameters in the same class. Method overriding is redefining a parent class method in a child class with the same signature.

Chapter 5: Polymorphism

Polymorphism means 'many forms'. It allows objects of different classes to be treated as objects of a common parent class, and the same method call can behave differently based on the object's actual class.

5.1 Compile-time Polymorphism (Method Overloading)

Same method name with different parameters in the same class.

Java

```
class Calculator {
    // Overloaded methods
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b)
        { return a + b;
    }

    int add(int a, int b, int c)
        { return a + b + c;
    }
}

// Usage
Calculator calc = new Calculator();
calc.add(5, 3);        // Calls first method
calc.add(5.5, 3.3);    // Calls second method
calc.add(1, 2, 3);     // Calls third method
```

Python

```
class Calculator:
    def add(self, a, b, c=None):
        if c is None:
            return a + b
        return a + b + c

# Usage
calc = Calculator()
print(calc.add(5, 3))       # 8
print(calc.add(5, 3, 2))    # 10
```

5.2 Runtime Polymorphism (Method Overriding + Dynamic Dispatch)

The actual method called is determined at runtime based on the object's type.

Java

```
class Animal {
    public void makeSound()
        { System.out.println("Some sound");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meow!");
    }
}

// Polymorphism in action
Animal myPet;
myPet = new Dog();
myPet.makeSound();  // Outputs: Woof!

myPet = new Cat();
myPet.makeSound();  // Outputs: Meow!
```

Python

```
class Animal:
    def make_sound(self):
        print("Some sound")

class Dog(Animal):
    def make_sound(self):
        print("Woof!")

class Cat(Animal):
    def make_sound(self):
        print("Meow!")
```

```
# Polymorphism in action
def animal_concert(animals):
    for animal in animals:
        animal.make_sound()

animals = [Dog(), Cat(), Dog()]
animal_concert(animals)
# Output: Woof! Meow! Woof!
```

5.3 Practice Questions

Question 1: Demonstrate polymorphism with a Shape class and Circle, Rectangle subclasses.

Python

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

# Polymorphism
shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

Question 2: What is the difference between compile-time and runtime polymorphism?

Answer: Compile-time polymorphism (method overloading) is resolved during compilation. Runtime polymorphism (method overriding) is resolved during program execution based on the actual object's type.

Chapter 6: Abstraction

Abstraction is the concept of hiding complex implementation details and showing only the essential features of an object. It helps reduce programming complexity.

6.1 Abstract Classes

Abstract classes cannot be instantiated and may contain abstract methods (methods without implementation).

Java

```
abstract class Shape
    { protected String color;

    // Concrete method
    public void setColor(String color) {
        this.color = color;
    }

    // Abstract method - no implementation
    public abstract double getArea();
    public abstract double getPerimeter();
}

class Circle extends Shape
    { private double radius;

    public Circle(double radius)
        { this.radius = radius;
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getPerimeter()
        { return 2 * Math.PI *
        radius;
    }
```

Python

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
    def __init__(self):
        self.color = ""

    def set_color(self, color):
        self.color = color

    @abstractmethod
    def get_area(self):
        pass

    @abstractmethod
    def get_perimeter(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        super().__init__()
        self.radius = radius

    def get_area(self):
        return 3.14 * self.radius ** 2

    def get_perimeter(self):
        return 2 * 3.14 * self.radius
```

6.2 Interfaces

An interface is a contract that specifies what methods a class must implement. A class can implement multiple interfaces.

Java

```
interface Drawable {
    void draw();
}

interface Resizable {
    void resize(double factor);
}

class Rectangle implements Drawable, Resizable {
    private double width, height;

    @Override
    public void draw() {
```

```
        System.out.println("Drawing rectangle");
    }

    @Override
    public void resize(double factor)
        { width *= factor;
        height *= factor;
    }
}
```

Python

```
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass

class Resizable(ABC):
    @abstractmethod
    def resize(self, factor):
        pass

class Rectangle(Drawable, Resizable):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def draw(self):
        print("Drawing rectangle")

    def resize(self, factor):
        self.width *= factor
        self.height *= factor
```

6.3 Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Abstract + Concrete	Abstract (default allowed)
Constructor	Yes	No
Multiple Inheritance	No	Yes
Use Case	'Is-a' relationship	'Can-do' capability

6.4 Practice Questions

Question 1: When should you use an abstract class vs an interface?

Answer: Use an abstract class when classes share common code and have an 'is-a' relationship. Use an interface when you want to define a contract that unrelated classes can implement, or when a class needs multiple behaviors.

Question 2: Create an abstract class Employee with abstract method calculate_salary().

Python

```python
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_salary(self):
        pass

class FullTimeEmployee(Employee):
    def __init__(self, name, monthly_salary):
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_salary(self):
        return self.monthly_salary

class PartTimeEmployee(Employee):
    def __init__(self, name, hourly_rate, hours_worked):
        super().__init__(name)
        self.hourly_rate = hourly_rate
        self.hours_worked = hours_worked

    def calculate_salary(self):
        return self.hourly_rate * self.hours_worked
```

Chapter 7: Constructors and Destructors

Constructors are special methods called when an object is created. Destructors are called when an object is destroyed. They help initialize and clean up resources.

7.1 Types of Constructors

1. Default Constructor

```
class Person {
private:
    string name;
    int age;

public:
    // Default constructor (no parameters)
    Person() {
        name = "Unknown";
        age = 0;
    }
};
```

2. Parameterized Constructor

C++

```
class Person {
private:
    string name;
    int age;

public:
    // Parameterized constructor
    Person(string n, int a) {
        name = n;
        age = a;
    }
};

// Usage
Person p1("Alice", 25);
```

Java

```
public class Person {
    private String name;
```

```
    private int age;

    public Person(String name, int age)
        { this.name = name;
        this.age = age;
    }
}

// Usage
Person p1 = new Person("Alice", 25);
```

Python

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Usage
p1 = Person("Alice", 25)
```

3. Copy Constructor

C++

```
class Person {
private:
    string name;
    int age;

public:
    // Copy constructor
    Person(const Person& other) {
        name = other.name;
        age = other.age;
    }
};

// Usage
Person p1("Alice", 25);
Person p2 = p1;  // Copy constructor called
```

7.2 Constructor Overloading

Java

```
public class Person
    { private String
    name;
```

```
    // Default constructor
    public Person() {
        this("Unknown", 0);  // Call parameterized constructor
    }

    // Constructor with name only
    public Person(String name) {
        this(name, 0);
    }

    // Full parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

7.3 Destructors

Destructors clean up resources when an object is destroyed.

C++

```
class FileHandler {
private:
    FILE* file;

public:
    FileHandler(const char* filename)
        { file = fopen(filename, "r");
    }

    // Destructor
    ~FileHandler() {
        if (file != nullptr) {
            fclose(file);
        }
    }
};
```

Python

```
class FileHandler:
    def __init__(self, filename):
        self.file = open(filename, 'r')

    def __del__(self):
```

```
        # Destructor
        if self.file:
            self.file.close()
            print("File closed")
```

7.4 Practice Questions

Question 1: Create a BankAccount class with constructors for different scenarios.

Python

```
class BankAccount:
    def __init__(self, account_number, owner, balance=0):
        self.account_number = account_number
        self.owner = owner
        self.balance = balance

    @classmethod
    def from_dict(cls, data):
        return cls(data['account_number'],
                   data['owner'],
                   data.get('balance', 0))

# Usage
account1 = BankAccount("12345", "Alice", 1000)
account2 = BankAccount("67890", "Bob")  # balance defaults to 0
```

Question 2: What is constructor chaining?

Answer: Constructor chaining is calling one constructor from another constructor in the same class. This helps avoid code duplication and provides multiple ways to initialize an object.

Chapter 8: Static and Final Keywords

Static and final keywords provide special behavior for variables, methods, and classes. Understanding them is crucial for writing efficient and secure code.

8.1 Static Keyword

Static members belong to the class rather than any specific object instance.

Static Variables (Class Variables)

Java

```
public class Counter {
    // Static variable - shared by all instances
    private static int count = 0;

    public Counter() {
        count++;  // Increment for each new object
    }

    public static int getCount()
        { return count;
    }
}

// Usage
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.getCount());  // Output: 2
```

Python

```
class Counter:
    count = 0  # Class variable

    def __init__(self):
        Counter.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

# Usage
c1 = Counter()
```

```
c2 = Counter()
print(Counter.get_count())  # Output: 2
```

Static Methods

Java

```
public class MathUtils {
    // Static method - belongs to class, not object
    public static int add(int a, int b) {
        return a + b;
    }

    public static double circleArea(double radius)
        { return Math.PI * radius * radius;
    }
}

// Usage - no object needed
int sum = MathUtils.add(5, 3);
double area = MathUtils.circleArea(5.0);
```

Python

```
class MathUtils:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def circle_area(radius):
        return 3.14 * radius ** 2

# Usage - no object needed
sum_val = MathUtils.add(5, 3)
area = MathUtils.circle_area(5.0)
```

8.2 Final Keyword

Final Variables (Constants)

Java

```
public class Constants {
    // Final variable - cannot be changed
    public static final double PI = 3.14159;
    public static final int MAX_USERS = 100;
}
```

```
// Usage
double area = Constants.PI * radius * radius;
```

Python

```
# Python doesn't have true constants, use convention
class Constants:
    PI = 3.14159
    MAX_USERS = 100

# Or use enum
from enum import Enum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3
```

Final Methods

```
public class Parent {
    // Final method - cannot be overridden
    public final void show() {
        System.out.println("Parent show");
    }
}

class Child extends Parent {
    // ERROR: Cannot override final method
    // public void show() { }
}
```

Final Classes

```
// Final class - cannot be extended
public final class String {
    // String class implementation
}

// ERROR: Cannot inherit from final class
// class MyString extends String { }
```

8.3 Practice Questions

Question 1: Create a Singleton class using static.

Java

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    // Private constructor
    private DatabaseConnection() { }

    // Static method to get instance
    public static DatabaseConnection getInstance()
        { if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

// Usage
DatabaseConnection db1 = DatabaseConnection.getInstance();
DatabaseConnection db2 = DatabaseConnection.getInstance();
// db1 and db2 are the same object
```

Python

```
class DatabaseConnection:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# Usage
db1 = DatabaseConnection()
db2 = DatabaseConnection()
print(db1 is db2)  # True - same object
```

Question 2: When should you use static methods?

Answer: Use static methods when the method doesn't depend on instance variables, when it performs a utility function, or when it makes sense to call the method without creating an object.

Chapter 9: Exception Handling

Exception handling allows programs to deal with unexpected situations gracefully without crashing. It's a crucial part of writing robust applications.

9.1 Try-Catch-Finally - Multi-Language

Java

```
try {
    int result = 10 / 0;  // This will throw ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} catch (Exception e) {
    System.out.println("An error occurred: " + e.getMessage());
} finally {
    // This block always executes
    System.out.println("Cleanup code here");
}
```

Python

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Cannot divide by zero: {e}")
except Exception as e:
    print(f"An error occurred: {e}")
else:
    # Runs if no exception occurred
    print(f"Result: {result}")
finally:
    # This block always executes
    print("Cleanup code here")
```

C++

```
try {
    int result = 10 / 0;
} catch (const std::runtime_error& e)
    { std::cout << "Runtime error: " <<
    e.what();
} catch (...) {
    // Catch all exceptions
    std::cout << "Unknown error occurred";
```

9.2 Throwing Exceptions

Java

```
public void withdraw(double amount) throws InsufficientFundsException
    { if (amount > balance) {
        throw new InsufficientFundsException("Not enough money");
    }
    balance -= amount;
}

// Custom exception
class InsufficientFundsException extends Exception
    { public InsufficientFundsException(String message)
    {
        super(message);
    }
```

Python

```
class InsufficientFundsError(Exception):
    pass

def withdraw(amount):
    if amount > balance:
        raise InsufficientFundsError("Not enough money")
    balance -= amount

# Usage
try:
    withdraw(1000)
except InsufficientFundsError as e:
    print(f"Error: {e}")
```

9.3 Common Exception Types

Exception	Java	Python
Null/None reference	NullPointerException	AttributeError/TypeError
Out of bounds	IndexOutOfBoundsException	IndexError
Invalid argument	IllegalArgumentException	ValueError
Division by zero	ArithmeticException	ZeroDivisionError

9.4 Practice Questions

Question 1: Write a function that safely converts a string to integer.

Python

```python
def safe_int_convert(value, default=0):
    try:
        return int(value)
    except (ValueError, TypeError):
        return default

# Usage
print(safe_int_convert("123"))      # 123
print(safe_int_convert("abc"))      # 0 (default)
print(safe_int_convert("abc", -1))  # -1
```

Question 2: What is the difference between checked and unchecked exceptions?

Answer: Checked exceptions (Java) are checked at compile-time and must be handled or declared. Unchecked exceptions occur at runtime and don't need explicit handling. Python and C++ don't have this distinction.

Chapter 10: Design Patterns

Design patterns are reusable solutions to common software design problems. They represent best practices evolved over time by experienced developers.

10.1 Creational Patterns

Singleton Pattern

Ensures a class has only one instance and provides global access to it.

Python

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

# Usage
s1 = Singleton()
s2 = Singleton()
print(s1 is s2)  # True
```

Factory Pattern

Creates objects without specifying the exact class to create.

Python

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
```

```
            return Cat()
        else:
            raise ValueError("Unknown animal type")

# Usage
dog = AnimalFactory.create_animal("dog")
print(dog.speak())  # Woof!
```

Builder Pattern

Constructs complex objects step by step.

Python

```
class Pizza:
    def __init__(self):
        self.size = None
        self.cheese = False
        self.pepperoni = False

class PizzaBuilder:
    def __init__(self):
        self.pizza = Pizza()

    def set_size(self, size):
        self.pizza.size = size
        return self

    def add_cheese(self):
        self.pizza.cheese = True
        return self

    def add_pepperoni(self):
        self.pizza.pepperoni = True
        return self

    def build(self):
        return self.pizza

# Usage
pizza = (PizzaBuilder()
         .set_size("Large")
         .add_cheese()
         .add_pepperoni()
         .build())
```

10.2 Structural Patterns

Adapter Pattern

Allows incompatible interfaces to work together.

Python

```python
# Target interface
class MediaPlayer:
    def play(self, filename):
        pass

# Adaptee
class AdvancedMediaPlayer:
    def play_mp4(self, filename):
        print(f"Playing mp4: {filename}")

# Adapter
class MediaAdapter(MediaPlayer):
    def __init__(self):
        self.advanced = AdvancedMediaPlayer()

    def play(self, filename):
        if filename.endswith(".mp4"):
            self.advanced.play_mp4(filename)

# Usage
player = MediaAdapter()
player.play("movie.mp4")
```

Decorator Pattern

Adds behavior to objects dynamically without affecting other objects.

Python

```python
class Coffee:
    def cost(self):
        return 5

    def description(self):
        return "Simple coffee"

class MilkDecorator:
    def __init__(self, coffee):
        self.coffee = coffee
```

```
    def cost(self):
        return self.coffee.cost() + 2

    def description(self):
        return self.coffee.description() + ", milk"

# Usage
coffee = Coffee()
coffee = MilkDecorator(coffee)
print(coffee.description())  # Simple coffee, milk
print(coffee.cost())  # 7
```

10.3 Behavioral Patterns

Observer Pattern

Defines a one-to-many dependency between objects.

Python

```
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

class Observer:
    def update(self, message):
        print(f"Received: {message}")

# Usage
subject = Subject()
observer1 = Observer()
observer2 = Observer()
subject.attach(observer1)
subject.attach(observer2)
subject.notify("Hello!")  # Both observers receive the message
```

Strategy Pattern

Defines a family of algorithms and makes them interchangeable.

Python

```
class PaymentStrategy:
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid {amount} using Credit Card")

class PayPalPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Paid {amount} using PayPal")

class ShoppingCart:
    def __init__(self, strategy):
        self.strategy = strategy

    def checkout(self, amount):
        self.strategy.pay(amount)

# Usage
cart = ShoppingCart(CreditCardPayment())
cart.checkout(100)
```

10.4 Practice Questions

Question 1: When would you use the Factory pattern?

Answer: Use the Factory pattern when you don't know beforehand what types of objects you'll need, when you want to delegate object creation to subclasses, or when you want to centralize object creation logic.

Question 2: Implement a Logger using Singleton pattern.

Python

```
class Logger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.logs = []
        return cls._instance

    def log(self, message):
```

```
        self.logs.append(message)
        print(f"LOG: {message}")

    def get_logs(self):
        return self.logs

# Usage - both loggers are the same object
logger1 = Logger()
logger2 = Logger()
logger1.log("First message")
print(logger2.get_logs())  # Shows "First message"
```

Chapter 11: SOLID Principles

SOLID is an acronym for five design principles that help make software designs more understandable, flexible, and maintainable.

11.1 S - Single Responsibility Principle

A class should have only one reason to change, meaning it should have only one job or responsibility.

Bad Example

```
class Employee {
    // Multiple responsibilities
    void calculatePay() { }
    void saveToDatabase() { }
    void generateReport() { }
}
```

Good Example

```
class Employee {
    // Only employee data
    private String name;
    private double salary;
}

class EmployeePayCalculator {
    double calculatePay(Employee emp) { }
}

class EmployeeRepository {
    void save(Employee emp) { }
}

class EmployeeReportGenerator {
    void generateReport(Employee emp) { }
}
```

11.2 O - Open/Closed Principle

Software entities should be open for extension but closed for modification.

Bad Example

```
class AreaCalculator {
    double calculate(Object shape) {
        if (shape instanceof Rectangle) {
            // Calculate rectangle area
        } else if (shape instanceof Circle) {
            // Calculate circle area
        }
        // Need to modify this method for each new shape!
    }
}
```

Good Example

```
interface Shape
    { double
    getArea();
}

class Rectangle implements Shape
    { public double getArea() {
        // Rectangle area calculation
    }
}

class Circle implements Shape
    { public double getArea() {
        // Circle area calculation
    }
}
```

11.3 L - Liskov Substitution Principle

Objects of a superclass should be replaceable with objects of subclasses without affecting correctness.

```
class Bird {
    void fly() { }
}

// Violates LSP - Penguin can't fly!
class Penguin extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException();
```

```
    }
}

// Better design
class Bird { }

class FlyingBird extends Bird
    { void fly() { }
}

class Penguin extends Bird {
    // No fly method needed
    void swim() { }
}
```

11.4 I - Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Split large interfaces into smaller ones.

Bad Example

```
interface Worker
    { void work();
    void eat();
}

class Robot implements Worker {
    public void work() { }
    public void eat() {
        // Robots don't eat! Empty implementation
    }
}
```

Good Example

```
interface Workable {
    void work();
}

interface Eatable
    { void eat();
}

class Human implements Workable, Eatable
    { public void work() { }
    public void eat() { }
```

```
}

class Robot implements Workable
    { public void work() { }
    // No eat method needed!
}
```

11.5 D - Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Bad Example

```
class MySQLDatabase
    { void connect() {
    }
}

// High-level module depends on low-level module
class UserService {
    private MySQLDatabase database;

    UserService() {
        this.database = new MySQLDatabase();
    }
```

Good Example

```
interface Database {
    void connect();
}

class MySQLDatabase implements Database
    { public void connect() { }
}

class PostgreSQLDatabase implements Database
    { public void connect() { }
}

// Depends on abstraction, not concrete class
class UserService {
    private Database database;

    UserService(Database database)
        { this.database = database;
    }
```

```
}

// Usage
UserService service = new UserService(new MySQLDatabase());
UserService service2 = new UserService(new PostgreSQLDatabase());
```

11.6 Practice Questions

Question 1: Explain why the Single Responsibility Principle is important.

Answer: SRP makes code easier to understand, test, and maintain. When a class has only one responsibility, changes to that responsibility only affect that class, reducing the risk of introducing bugs elsewhere.

Question 2: Give an example of violating and following the Open/Closed Principle.

Answer: Violation: Adding if-else statements for each new shape type. Following: Using polymorphism where each shape implements a common interface, allowing new shapes to be added without modifying existing code.

Chapter 12: Interview Preparation

This chapter contains commonly asked OOP interview questions and their answers. Understanding these concepts thoroughly will help you succeed in technical interviews.

12.1 Common OOP Interview Questions

Q1: What is the difference between abstraction and encapsulation?

Answer: Abstraction hides implementation details and shows only functionality (what an object does). Encapsulation wraps data and code together and controls access (how data is accessed). Abstraction is achieved through abstract classes and interfaces. Encapsulation is achieved through access modifiers.

Q2: Explain the difference between 'is-a' and 'has-a' relationships.

Answer: 'Is-a' represents inheritance (Dog is-a Animal). 'Has-a' represents composition or aggregation (Car has-a Engine). 'Is-a' is implemented through class inheritance, while 'has-a' is implemented through member variables.

Q3: What is method overloading vs method overriding?

Answer: Method overloading is having multiple methods with the same name but different parameters in the same class (compile-time polymorphism). Method overriding is redefining a parent class method in a child class with the same signature (runtime polymorphism).

Q4: Why is Java not purely object-oriented?

Answer: Java is not purely object-oriented because it uses primitive types (int, float, char, etc.) which are not objects. However, Java provides wrapper classes (Integer, Float, Character) to treat primitives as objects when needed.

Q5: What is the diamond problem in inheritance?

Answer: The diamond problem occurs when a class inherits from two classes that both inherit from the same parent class, creating ambiguity about which parent method to use. Java solves this by not allowing multiple inheritance of classes. C++ allows it but requires explicit qualification. Python uses the Method Resolution Order (MRO).

Q6: Explain the difference between abstract class and interface.

Answer: Abstract classes can have both abstract and concrete methods, can have constructors,

and support single inheritance. Interfaces (pre-Java 8) only have abstract methods, no constructors, and a class can implement multiple interfaces. Modern interfaces can have default and static methods.

Q7: What is the purpose of the 'super' keyword?

Answer: The 'super' keyword is used to call parent class methods or constructors. It's used to access overridden methods, call parent constructors, and access parent class variables that are hidden by child class variables.

Q8: What is composition over inheritance?

Answer: Composition over inheritance is a design principle that suggests using composition (has-a relationship) instead of inheritance (is-a relationship) when possible. It's more flexible because you can change behavior at runtime and avoid the fragility of deep inheritance hierarchies.

12.2 Coding Problems

Problem 1: Design a Parking Lot System

Python

```python
from enum import Enum

class VehicleType(Enum):
    MOTORCYCLE = 1
    CAR = 2
    TRUCK = 3

class Vehicle:
    def __init__(self, license_plate, vehicle_type):
        self.license_plate = license_plate
        self.vehicle_type = vehicle_type

class ParkingSpot:
    def __init__(self, spot_id, spot_type):
        self.spot_id = spot_id
        self.spot_type = spot_type
        self.vehicle = None

    def park(self, vehicle):
        if self.vehicle is None:
            self.vehicle = vehicle
            return True
        return False
```

```
    def remove_vehicle(self):
        self.vehicle = None

class ParkingLot:
    def __init__(self):
        self.spots = []

    def add_spot(self, spot):
        self.spots.append(spot)

    def find_available_spot(self, vehicle_type):
        for spot in self.spots:
            if spot.vehicle is None and spot.spot_type == vehicle_type:
                return spot
        return None
```

Problem 2: Design a Library Management System

Python

```
class Book:
    def __init__(self, isbn, title, author):
        self.isbn = isbn
        self.title = title
        self.author = author
        self.is_available = True

class Member:
    def __init__(self, member_id, name):
        self.member_id = member_id
        self.name = name
        self.borrowed_books = []

    def borrow_book(self, book):
        if book.is_available:
            book.is_available = False
            self.borrowed_books.append(book)
            return True
        return False

    def return_book(self, book):
        if book in self.borrowed_books:
            book.is_available = True
            self.borrowed_books.remove(book)
            return True
        return False
```

```
class Library:
    def __init__(self):
        self.books = []
        self.members = []

    def add_book(self, book):
        self.books.append(book)

    def register_member(self, member):
        self.members.append(member)
```

12.3 Tips for OOP Interviews

- Understand the four pillars thoroughly and be able to give real-world examples
- Practice designing systems using OOP principles
- Know when to use composition vs inheritance
- Be familiar with common design patterns
- Understand SOLID principles and their applications
- Practice coding problems that involve class design

Final Words

Mastering Object-Oriented Programming takes practice and patience. Start with small projects, apply the principles you've learned, and gradually tackle more complex designs. Remember, good OOP design is about creating code that is maintainable, extensible, and easy to understand.

Good luck with your interviews and happy coding!

- Mohammad Faiz

Appendix A: Language Syntax Comparison

Quick reference guide comparing syntax across C++, Java, and Python.

A.1 Class Definition

C++

```
class MyClass {
private:
    int privateVar;
public:
    int publicVar;
    MyClass(int val) { privateVar = val; }
    void method() { }
};
```

Java

```
public class MyClass
    { private int
    privateVar; public int
    publicVar;
    public MyClass(int val) { this.privateVar = val; }
    public void method() { }
```

Python

```
class MyClass:
    def __init__(self, val):
        self.__private_var = val
        self.public_var = 0
    def method(self): pass
```

A.2 Inheritance

C++

```
class Child : public Parent { };
```

Java

```
class Child extends Parent { }
```

Python

```
class Child(Parent): pass
```

A.3 Method Overriding

C++

```
class Parent {
    virtual void show() { }
};
class Child : public Parent
    { void show() override { }
};
```

Java

```
class Parent {
    void show() { }
}
class Child extends Parent {
    @Override
    void show() { }
}
```

Python

```
class Parent:
    def show(self): pass
class Child(Parent):
    def show(self): pass
```

A.4 Abstract Class / Interface

Java

```
abstract class Shape
    { abstract double
    getArea();
}
interface Drawable {
    void draw();
```

Python

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def get_area(self): pass
```

A.5 Common Operations Table

Operation	C++	Java	Python
Constructor	ClassName() {}	ClassName() {}	__init__(self)
Inheritance	class C : public P	class C extends P	class C(P)
Super call	Parent::method()	super.method()	super().method()
This/self	this	this	self

Appendix B: OOP Cheat Sheet

Quick reference for OOP concepts and best practices.

B.1 The Four Pillars

```
ENCAPSULATION:
- Bundle data and methods together
- Control access with private/protected/public
- Use getters/setters for controlled access

INHERITANCE:
- 'Is-a' relationship
- Code reuse through parent classes
- Method overriding for customization

POLYMORPHISM:
- Same interface, different implementations
- Compile-time: Method overloading
- Runtime: Method overriding

ABSTRACTION:
- Hide implementation details
- Show only essential features
- Use abstract classes and interfaces
```

B.2 Access Modifiers

Modifier	C++	Java	Python
Private	private:	private	__var
Protected	protected:	protected	_var
Public	public:	public	var

B.3 SOLID Principles Summary

```
S - Single Responsibility Principle:
   One class, one responsibility

O - Open/Closed Principle:
   Open for extension, closed for modification
```

```
L - Liskov Substitution Principle:
   Subtypes must be substitutable for base types

I - Interface Segregation Principle:
   Split large interfaces into smaller ones

D - Dependency Inversion Principle:
   Depend on abstractions, not concretions
```

B.4 Design Patterns Quick Reference

```
CREATIONAL:
- Singleton: One instance only
- Factory: Create objects without specifying class
- Builder: Construct complex objects step by step

STRUCTURAL:
- Adapter: Make incompatible interfaces work
- Decorator: Add behavior dynamically
- Composite: Tree structure of objects

BEHAVIORAL:
- Observer: One-to-many dependency
- Strategy: Interchangeable algorithms
- Command: Encapsulate requests as objects
```

B.5 Common Mistakes to Avoid

- God Class: A class that does too much - violates SRP
- Deep Inheritance: Too many levels of inheritance - hard to maintain
- Premature Abstraction: Abstracting before understanding the problem
- Tight Coupling: Classes too dependent on each other
- Ignoring Encapsulation: Making everything public

Multi-Language Edition

C++ | Java | Python | JavaScript

Mohammad Faiz

© 2026 All Rights Reserved